Leer al compilador

Cómo entender y arreglar los errores de compilación en C

Cuando `gcc` se queja, no está diciendo «tu programa está mal». Está diciendo **exactamente en qué línea, y por qué, ha dejado de entenderlo**. Es la mejor pista que vas a tener. El problema es que casi nadie enseña a leerla, y lo que hace la mayoría de la gente es mirar la última línea, ver que dice `error`, y ponerse a cambiar cosas al azar.

Estos apuntes van de lo contrario: leer despacio, entender qué dice, y arreglar lo que dice. Al final hay nueve programas rotos para practicar.

---

0. Antes de nada: dos cosas que configurar

Compila siempre con `-Wall`. Sin esa opción, gcc se calla la mitad de lo que sabe.

```
gcc -Wall -o programa fichero.c
```

Fuerza los mensajes en inglés. Si tu sistema está en español, gcc traduce los mensajes, y entonces no coinciden ni con estos apuntes ni con nada que encuentres buscando. Pon esto al principio de cada sesión (o en tu `.bashrc`):

```
export LANG=C
```

---

1. Las tres fases: saber *quién* se queja

Cuando escribes `gcc a.c b.c`, en realidad pasan tres cosas seguidas, y **cada una se queja de forma distinta**. Saber en qué fase estás te ahorra la mitad del trabajo.

FASE	QUÉ HACE	CÓMO SE RECONOCE SU ERROR
Preprocesador	Resuelve los `#include` y `#define`	`fatal error: pila.h: No such file or directory`
Compilador	Traduce cada `.c` a código máquina, **por separado**	`fichero.c:12:5: error: ...` — siempre con fichero, línea y columna
Enlazador (`ld`)	Junta todos los `.c` compilados en un ejecutable	`undefined reference to 'apilar'` — **sin número de línea**

La distinción importante es la última. Cuando el enlazador se queja, **tu código C está bien**: cada fichero por separado ha compilado. Lo que falla es que al juntar las piezas falta una, o sobra una. Casi siempre es **la orden de `gcc`**: te has dejado un `.c` sin poner, o has puesto uno dos veces.

---

2. Anatomía de un mensaje

Esto es lo que dice gcc cuando falta una `#include <string.h>`:

```
ejercicios/03_no_declarado.c: In function 'main':
ejercicios/03_no_declarado.c:8:21: error: implicit declaration of function 'strlen'
    8 |     for (i = 0; i < strlen(palabra); i++) {
      |                     ^~~~~~
ejercicios/03_no_declarado.c:3:1: note: include '<string.h>' or provide a
declaration of 'strlen'
    2 | #include <stdio.h>
  +++ |+#include <string.h>
```

Léelo por partes:

TROZO	QUÉ ES	
`In function 'main':`	Contexto: dónde estaba gcc cuando pasó.	
`03_no_declarado.c:8:21:`	**Fichero, línea, columna.** La línea 8, el carácter 21.	
`error:`	Gravedad. Puede ser `error`, `warning` o `note`.	
`implicit declaration of function 'strlen'`	El mensaje. Está en el catálogo de más abajo.	
`8 \	for (i = 0; ...`	La línea de tu código, tal cual.	
`^~~~~~`	**El acento circunflejo señala el sitio exacto.** Míralo siempre.	
`note: include '<string.h>'`	Una **nota**: gcc te está diciendo cómo arreglarlo.	
`+++ \	+#include <string.h>`	Una **sugerencia de cambio**: la línea que gcc añadiría.	

Fíjate en las dos últimas filas. **gcc muy a menudo te dice la solución.** `did you mean 'contador'?`, `did you mean to use '->'?`, `include '<string.h>'`… Cuando veas un `note:` o un `did you`

`mean`, léelo antes que nada.

---

3. Las cinco reglas

Regla 1 — Lee el PRIMER error. Solo el primero.

Un único fallo puede provocar una cascada de veinte errores, porque a partir de ese punto gcc ya no entiende nada de lo que sigue. Los errores 2 a 20 **no existen**: desaparecerán solos cuando arregles el primero.

Así que: arregla el primero, **vuelve a compilar**, y mira otra vez. Nunca intentes arreglar la lista entera de una vez.

(Ejercicio 07: un solo punto y coma que falta produce once errores.)

Regla 2 — La línea que dice gcc es donde se dio cuenta, no siempre donde está el fallo.

Si te dejas un `;` al final de la línea 5, gcc no lo nota hasta que empieza a leer la línea 6 y se encuentra algo que no esperaba. El error dirá **línea 6**. El fallo está en la 5.

Regla práctica: si en la línea que señala gcc no ves nada raro, **mira la línea anterior**. Y si el error está en la última línea del fichero (`at end of input`), es que falta una llave de cierre en alguna parte.

(Ejercicios 01 y 02.)

Regla 3 — Un aviso es un error que todavía no ha explotado.

`warning` significa que el programa **compila y se ejecuta**. Pero mira este caso real: un programa con cuatro avisos, que compila, se ejecuta… y hace esto:

```
7 es mayor que 10
El maximo es 42
Hola, 2063728681
```

Tres de los cuatro avisos eran fallos de verdad. **Con `-Wall`, trata cada aviso como un error.** En esta asignatura, un programa con avisos no está terminado.

(Ejercicio 06.)

Regla 4 — Si dice `undefined reference`, tu código está bien: mira la orden.

Es el enlazador. Has escrito `gcc main.c` y te has dejado el `pila.c`. O al revés: has hecho `#include "pila.c"` (con `.c`, en vez de `.h`) y ahora las funciones están definidas dos veces.

(Ejercicios 08 y 09.)

Regla 5 — Cuando gcc sugiere algo, suele tener razón.

`did you mean 'contador'?` — sí, querías decir `contador`. `did you mean to use '->'?` — sí. No siempre acierta, pero acierta tanto que merece la pena leerlo antes de pensar.

---

4. Catálogo de mensajes

Los mensajes están copiados literalmente de gcc 13. Para cada uno: qué significa, por qué suele pasar, y cómo se arregla.

4.1 Errores de sintaxis — gcc ha perdido el hilo

`expected ',' or ';' before 'int'` (o `before 'return'`, `before '}'`…)

Falta un punto y coma **en la línea anterior** a la señalada. Lo que va después de `before` es lo primero que gcc ha encontrado tras el hueco; lo que va antes es lo que esperaba encontrar.

`expected declaration or statement at end of input`

Se ha acabado el fichero y hay una llave `{` que nunca se ha cerrado. La línea que señala es la última; el fallo está en algún `}` que falta más arriba. Cuenta llaves, o deja que el editor te las empareje.

`expected ';' before 'int'` justo después de `} Punto`

Un `typedef struct { ... } Nombre` termina en **punto y coma**. Olvidarlo es un clásico, y como está al principio del fichero, produce una cascada: todo lo que use `Punto` a partir de ahí fallará con `unknown type name 'Punto'`. Regla 1.

4.2 Errores de declaración — gcc no sabe qué es esa palabra

`'contadr' undeclared (first use in this function); did you mean 'contador'?`

Has escrito un nombre que no existe. Casi siempre es una errata, y gcc te propone la corrección. Si no es una errata, es que la variable está declarada en otro sitio (dentro de otra función, o más abajo).

`implicit declaration of function 'strlen'`

Llamas a una función que gcc no conoce. Dos causas: **falta un `#include`** (para `strlen` es `string.h`, para `printf` es `stdio.h`, para `malloc` es `stdlib.h`, para `bool` es `stdbool.h`), o es una función tuya que está definida **más abajo** de donde la usas — entonces pon su prototipo arriba, o cambia el orden.

`unknown type name 'Punto'`

Igual que el anterior, pero con un tipo. O falta el `#include` de la cabecera donde está el `typedef`, o el `typedef` tiene un error de sintaxis y gcc no ha llegado a entenderlo (Regla 1).

4.3 Errores de tipos y punteros — los de esta asignatura

`'nueva' is a pointer; did you mean to use '->'?`

Has escrito `nueva.elem` y `nueva` es un puntero. Con punteros se usa `->`.

`invalid type argument of '->' (have 'celdaPila')`

Lo contrario: has escrito `copia->sig` y `copia` **no** es un puntero, es la estructura directamente. Con estructuras se usa `.`.

Regla mnemotécnica: `->` es «sigue la flecha hasta la estructura». Si ya tienes la estructura en la mano, no hay flecha que seguir.

`passing argument 1 of 'nuevaPila' from incompatible pointer type` `note: expected 'celdaPila **' but argument is of type 'tipoPila'`

El error más frecuente de las prácticas. La función quiere la **dirección** de tu variable (`&pila`) y le has pasado la variable (`pila`), o al revés. Lee el `note`: dice literalmente qué esperaba y qué le has dado. Cuenta los asteriscos: si espera uno más de los que le das, falta un `&`; si espera uno menos, sobra.

`'pila' is used uninitialized`

Estás leyendo una variable antes de haberle dado ningún valor. En las prácticas suele ir de la mano del anterior: si llamas a `nuevaPila(pila)` en vez de `nuevaPila(&pila)`, la variable nunca se inicializa.

`request for member 'x' in something not a structure or union`

Estás usando `.x` sobre algo que no es una estructura. O la variable no es del tipo que crees, o —muy a menudo— el tipo está roto por un error anterior (Regla 1).

4.4 Avisos que son errores

`control reaches end of non-void function`

Una función que debe devolver algo tiene un camino por el que sale **sin** `return`. Casi siempre: un `if` con `return` dentro y ningún `return` en el `else`. El programa compila, y esa función devuelve **basura** cuando pasa por ese camino. Este aviso provocó un fallo real en las soluciones de la asignatura que solo se manifestaba al compilar con `-O2`.

`suggest parentheses around assignment used as truth value`

Has escrito `if (x = 5)` en vez de `if (x == 5)`. Con un `=` estás asignando, no comparando, y la condición es siempre cierta. **Ojo:** gcc solo avisa si la asignación es la condición entera. Si está dentro de un `&&` o un `||` —`if (a == 1 && b = 2)`— **no avisa**, y ese fallo también apareció en las soluciones de la asignatura. Cuando escribas una condición, cuenta los iguales.

`format '%d' expects argument of type 'int', but argument 2 has type 'char *'`

En el `printf` has puesto un `%d` y le pasas una cadena (o al revés). Compila, y en ejecución imprime un número sin sentido. gcc te señala el `%` culpable y te sugiere el correcto.

`unused variable 'sinUsar'`

Has declarado una variable que no usas. **Este es inofensivo**: no rompe nada. Pero arréglalo igual, porque un programa que compila limpio es la única forma de que un aviso nuevo se vea. Si te acostumbras a ignorar avisos, un día ignorarás uno de los de arriba.

4.5 Errores del enlazador — la orden está mal, no el código

`undefined reference to 'nuevaPila'` (y normalmente varios seguidos)

Usas una función que está declarada en un `.h` pero cuya definición está en un `.c` que **no has puesto en la orden de compilar**. El código es correcto. Añade el fichero:

```
gcc -Wall main.c pila.c
```

Fíjate en que el mensaje no lleva número de línea y viene de `/usr/bin/ld`: es la firma del enlazador.

`undefined reference to 'main'`

Has compilado solo la librería, sin el fichero que tiene el `main`. O has compilado un fichero de pruebas al que le falta el `main`.

`multiple definition of 'nuevaPila'; first defined here`

La misma función está definida dos veces. Casi siempre es porque has escrito `#include "pila.c"` en vez de `#include "pila.h"`: al incluir el `.c`, su contenido se copia dentro de `main.c`, y luego además compilas `pila.c` por separado. **Se incluyen los `.h`, se compilan los `.c`.**

`fatal error: pila.h: No such file or directory`

Este es del preprocesador. El fichero no está donde gcc lo busca: o está en otra carpeta, o el nombre no coincide exactamente (mayúsculas incluidas).

---

5. Cómo practicar

En la carpeta `ejercicios/` hay nueve programas que **no compilan**, cada uno con uno o varios fallos de los del catálogo. Cada fichero dice al principio qué debería imprimir si funcionara.

```
./comprobar.sh          ver el estado de todos
./comprobar.sh 03       compilar solo el 03 y ver lo que dice gcc
./reiniciar.sh 03       dejar el 03 como estaba, si lo has liado
```

Un ejercicio está resuelto cuando compila **sin errores ni avisos** y además imprime lo que tiene que imprimir. El comprobador te lo dice.

El método, siempre el mismo:

1. `./comprobar.sh NN` y lee **solo el primer mensaje**.
2. Búscalo en el catálogo. Si hay un `note` o un `did you mean`, hazle caso.
3. Arregla eso y solo eso.
4. Vuelve al paso 1 hasta que salga en verde.

	QUÉ PRACTICA
01	Regla 2: el error está en la línea anterior
02	Regla 2: `at end of input` = falta una llave
03	Un `#include` que falta y una errata; gcc sugiere las dos soluciones
04	`.` y `->`
05	`&` que falta y `&` que sobra
06	Regla 3: cuatro avisos, tres de ellos fallos reales
07	Regla 1: un punto y coma, once errores
08	Regla 4: el código está bien, la orden no
09	Regla 4: `#include "pila.c"`

Cuando termines los nueve, coge tus propias prácticas y compílalas con `-Wall`. Lo que salga, ya sabes leerlo.